

Group 1: Reinforcement Learning in a High-Variance, Combinatorial World

Aidan Jared - ajar667

Daoyu Zhang - dzha977

Dominicus Nararya - dnar278

Jason Kong - jkon763

Thomas Clunie - tclu830

Yuanyuan Yu - yyu119



Research Question

Does Hierarchical Reinforcement Learning improve learning in sparse reward environments

Reinforcement Learning requires dense rewards to effectively learn and lacks strong long-term planning.

Hierarchical Reinforcement Learning intends to solve this by dividing into multiple problems which different models learn

Research Outline

Design Q-Model [1] as baseline and compare against FeUdal Model [2]

Train Model on Sparse Reward Environments in Gymnasium [3]

Test models and run scores through *rliale* [4] to generate evaluation metrics and measure performance

Convolution

Convolution

Fully connected

Fully connected

No input



Methodology: Environment

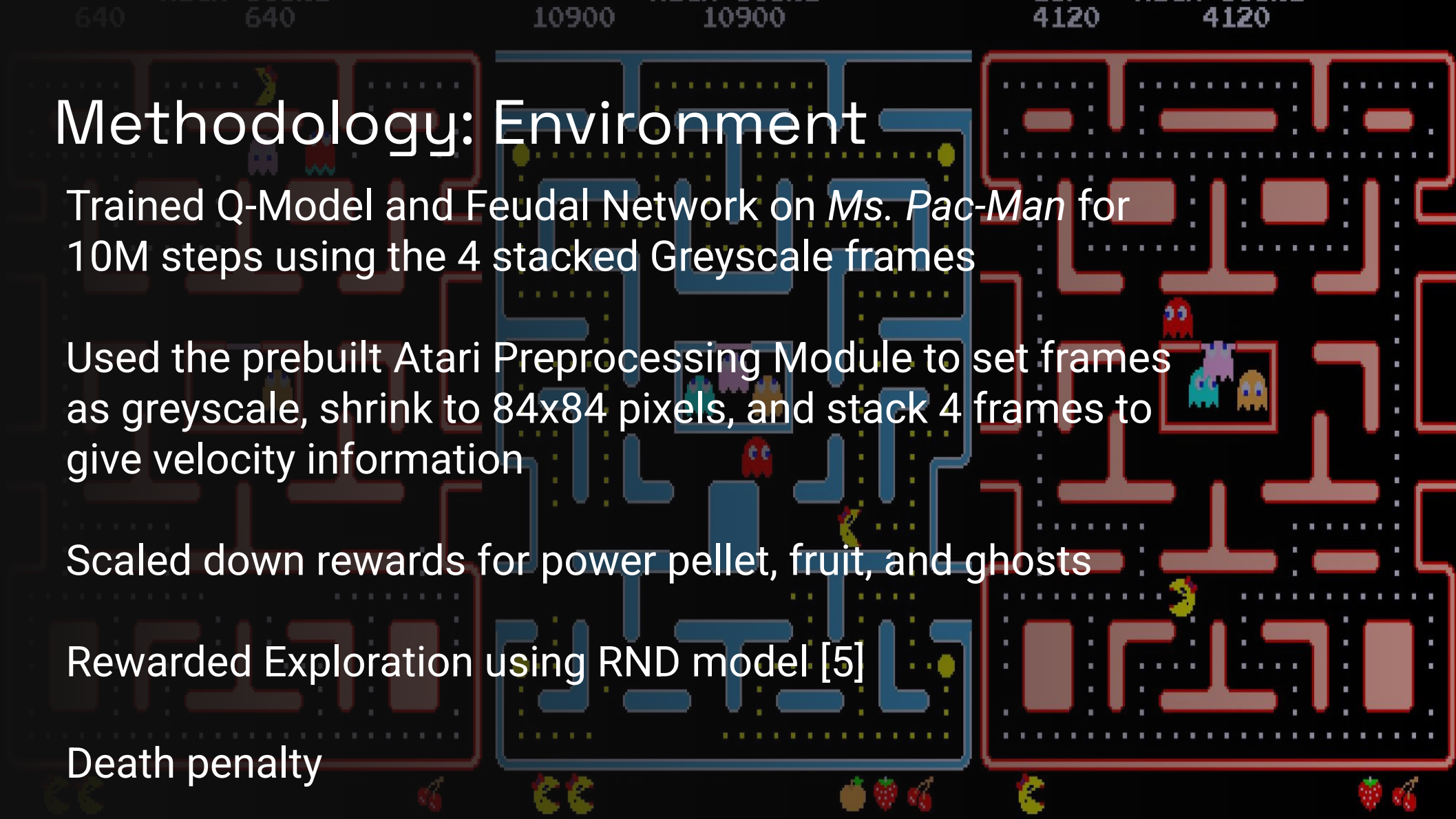
Trained Q-Model and Feudal Network on *Ms. Pac-Man* for 10M steps using the 4 stacked Greyscale frames

Used the prebuilt Atari Preprocessing Module to set frames as greyscale, shrink to 84x84 pixels, and stack 4 frames to give velocity information

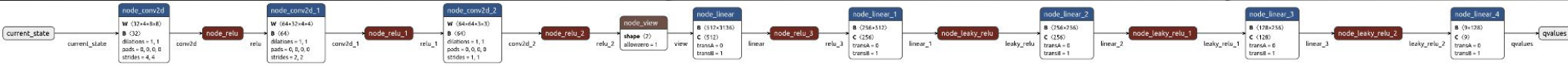
Scaled down rewards for power pellet, fruit, and ghosts

Rewarded Exploration using RND model [5]

Death penalty



Methodology: Q-Model



Built Deep Q-Network [6], tries to predict the future cumulative reward for each action

Used Double Q-Learning [7] to stabilize training, make predictions twice to reduce noise

Learns off-policy, and randomly samples from queue of past states every 4 steps

Hidden of 512, lr of 1e-4, γ of .99

$$\left(\boxed{r + \gamma \max_{a'} Q(s', a'; \theta_i^-)} - \boxed{Q(s, a; \theta_i)} \right)^2$$

Target

Prediction

Target Network

Prediction Network

Input

Methodology: Feudal Network

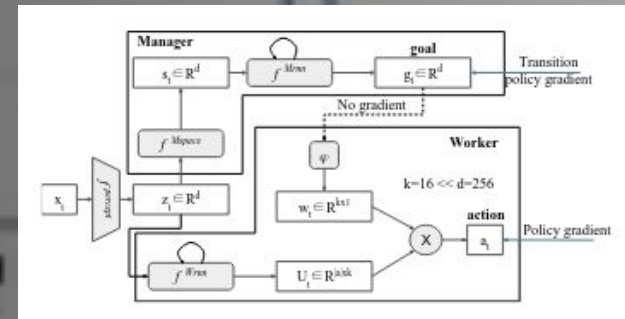
Uses a Manager to Produce Goals and a Worker to produce a probability distribution of actions

Worker uses a LSTM and a Critic

Manager uses a *Dilated LSTM* and Critic (only attends to small portion of hidden state)

Goals are weights on the workers actions, similar to attention mechanisms

Used Manager hidden of 256, Worker Hidden of 16
 γ_m of .99, γ_w of .95, time horizon of 10, dilation of 10, α of .1, entropy coef of .01



Solutions

Shrink Replay buffer and store values as ints

Big solution was to use tanh on the goals inside the Worker to keep directional information but shrink effect

Tried adding a penalty for negative cosine similarity as well as including Goal Entropy in loss calculation

Scaling down critic loss to lower effect on loss calculation

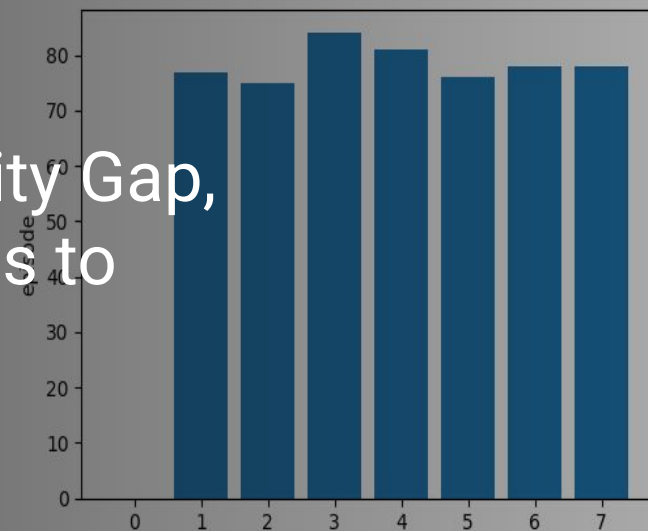
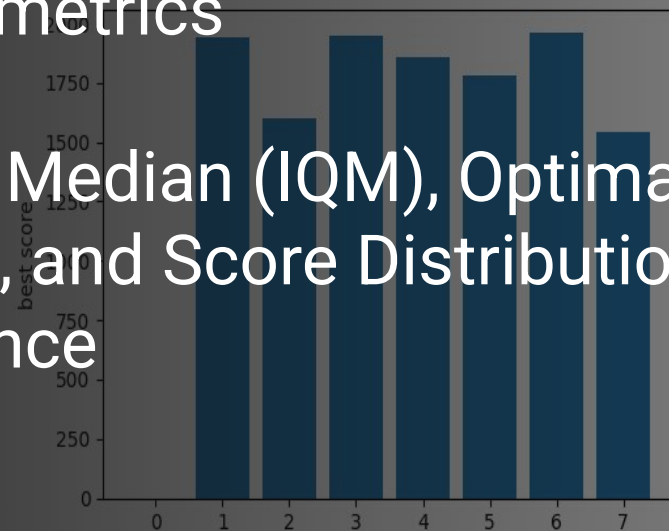
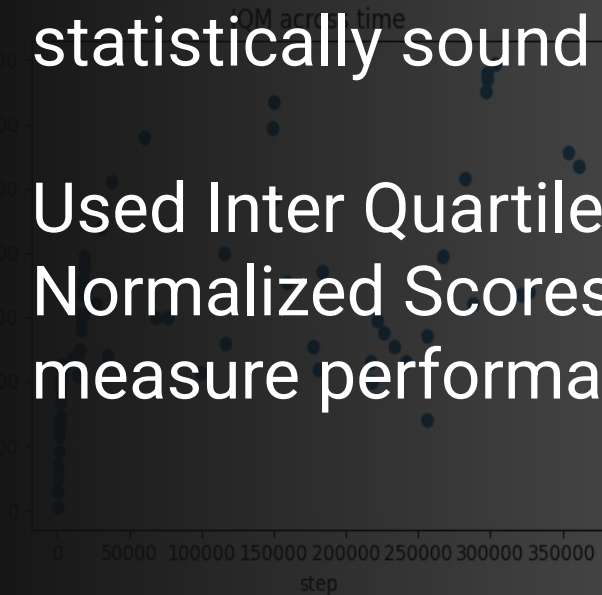
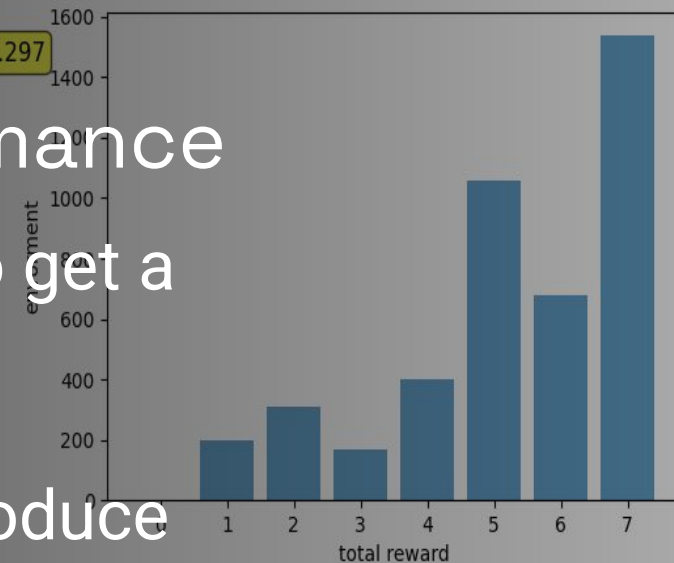
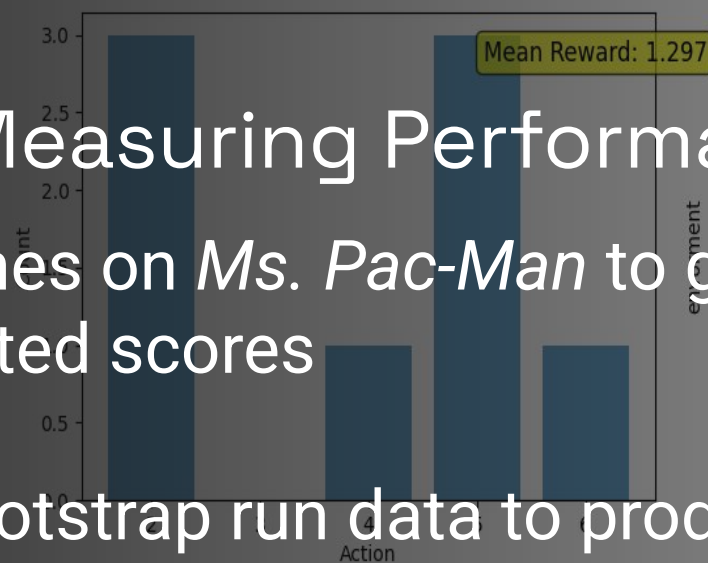
Changing from 1 RGB image to 4 grayscale as input

Methodology: Measuring Performance

Ran models 100 times on *Ms. Pac-Man* to get a spread of uncorrelated scores

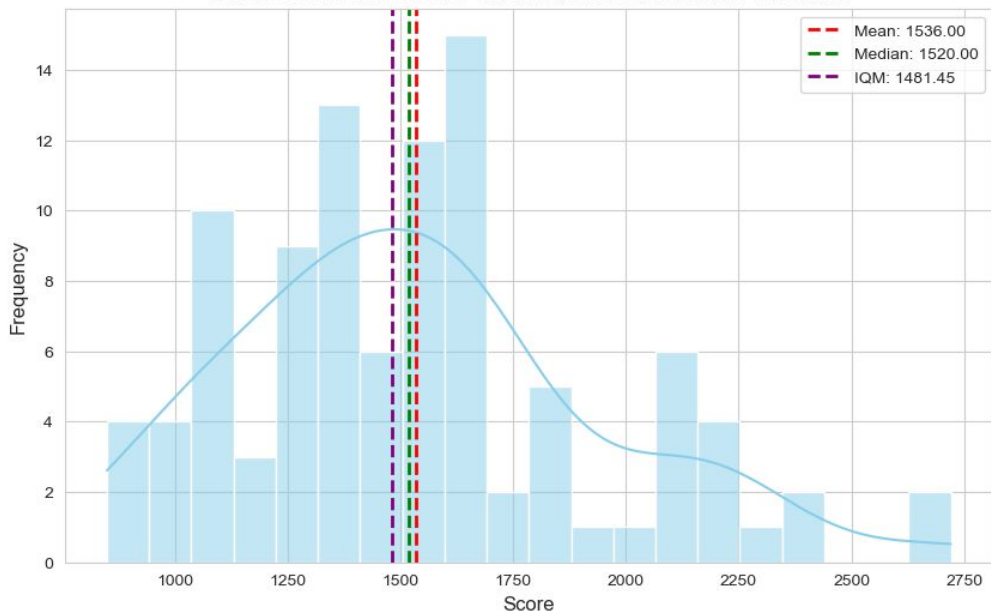
We use *reliable* to bootstrap run data to produce statistically sound metrics

Used Inter Quartile Median (IQM), Optimality Gap, Normalized Scores, and Score Distributions to measure performance

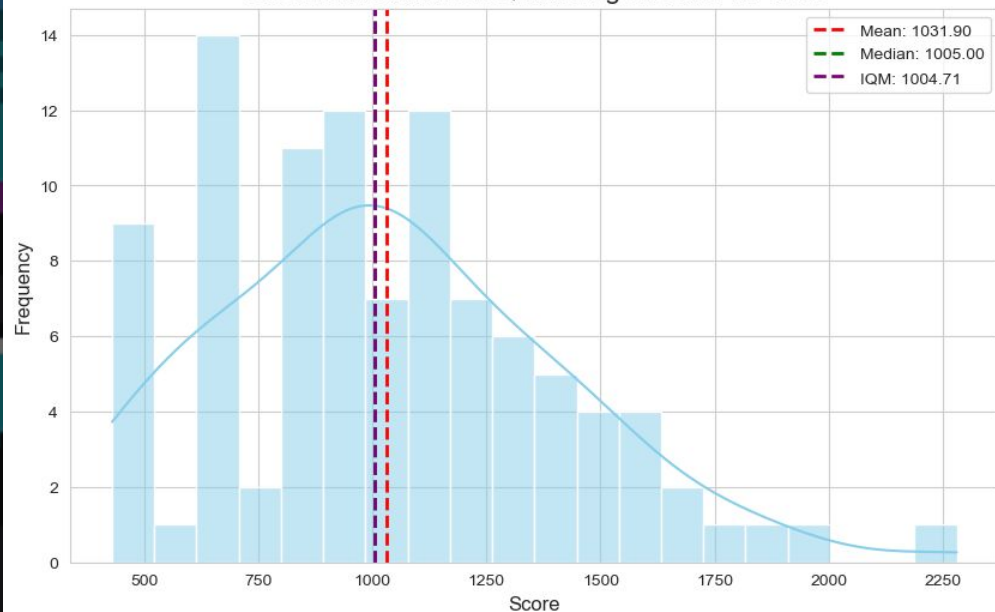


Results: Score Distributions

Score Distribution of Feudal Network on Ms. Pac-Man

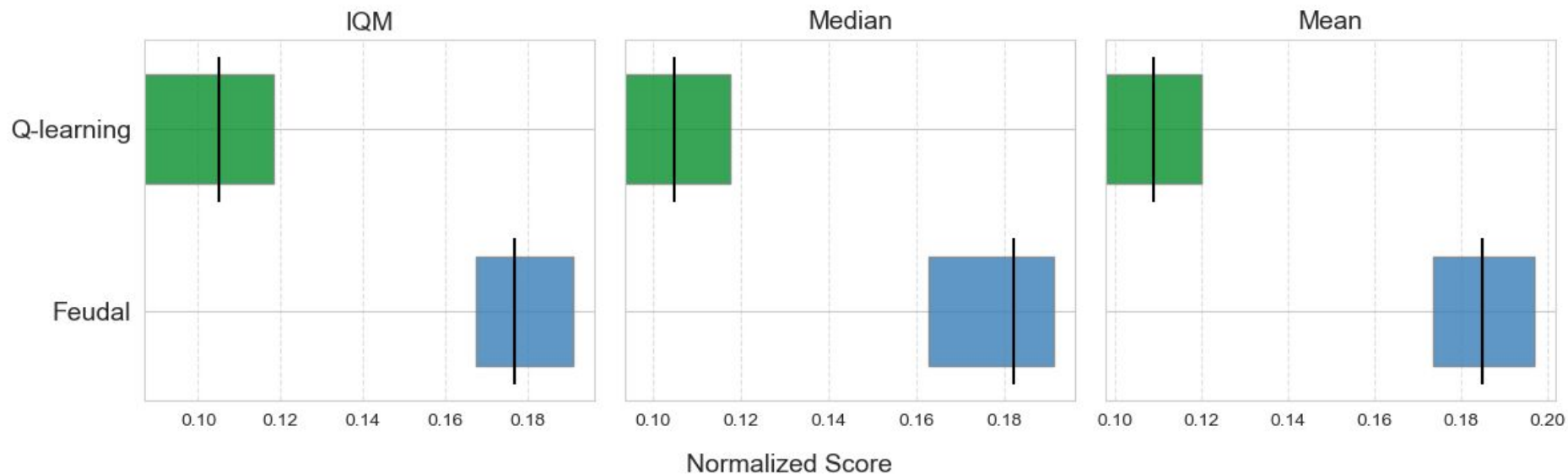


Score Distribution of Q-learning on Ms. Pac-Man

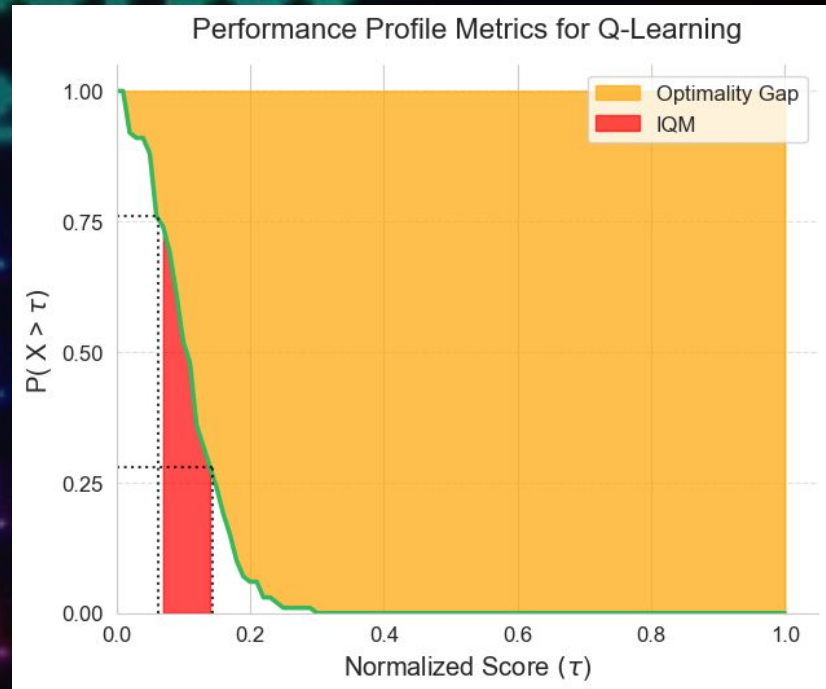
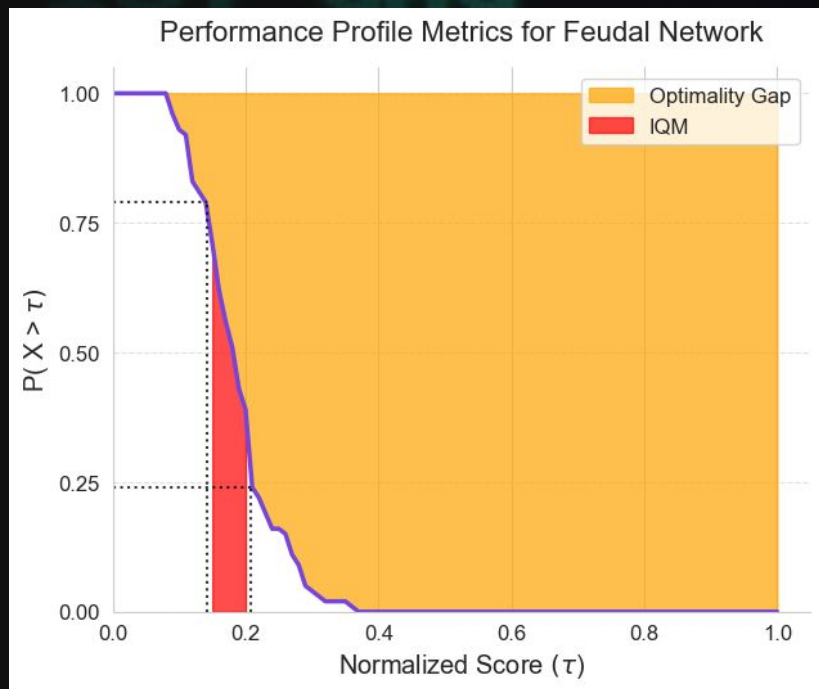


Results: Performance Comparison

Performance Comparison on Ms. Pac-Man



Results: Performance Comparison



- $P(X > \tau)$ represents the probability a single run of model will achieve a score X that is greater than a specific performance threshold τ
- Normalized score, 0 is random, 1 is Human.

Conclusion

In the single testing environment, the Feudal Model outperforms Q-Learning after training for 10e6 steps

Hierarchical goal-setting improves exploration and long-term planning

Underperforms Human ($\tau = 1$) but this might be due to short training time

Future Work

Train on more steps to get better performance on both models

Implement attention mechanism between goals and worker

Investigate using non spatial goals so architecture can be used in non-CNN contexts

Training model on more difficult games such as *Montezuma's Revenge* or *Seaquest*

Contributions

Aidan Jared oversaw the project, designed the presentations, designed the feudal model, reward metrics, training and testing environments, trained the models.

Daoyu Zhang participated in the collection of literature and research materials and the initial cleaning of the dataset.

Thomas Clunie designed the qmodel and trained/tested it on multiple hyperparameters

Yuanyuan Yu participated in the collection of literature and research materials.

Dominicus Nararya processed Model score data, created data visualisations, utilized *rl* for robust model evaluation, contributed to the design of reward metrics, and contributed to presentations.

References

[1] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, vol. 8, no. 3, pp. 279–292, 1992, doi: 10.1007/BF00992698

[2] A. S. Vezhnevets et al., “FeUdal Networks for Hierarchical Reinforcement Learning,” arXiv preprint arXiv:1703.01161, 2017, [Online]. Available: <https://arxiv.org/abs/1703.01161>

[3] M. Towers et al., “Gymnasium: A Standard Interface for Reinforcement Learning Environments,” arXiv preprint arXiv:2407.17032, 2024.

[4] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, “Deep Reinforcement Learning at the Edge of the Statistical Precipice,” CoRR, 2021, [Online]. Available: <https://arxiv.org/abs/2108.13264>

[5] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, “Exploration by Random Network Distillation,” arXiv preprint arXiv:1810.12894, 2018, [Online]. Available: <https://arxiv.org/abs/1810.12894>

[6] V. Mnih et al., “Playing Atari with Deep Reinforcement Learning,” CoRR, 2013, [Online]. Available: <http://arxiv.org/abs/1312.5602>

[7] H. van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-learning,” CoRR, 2015, [Online]. Available: <http://arxiv.org/abs/1509.06461>

Questions?

https://github.com/Aidan-Jared/COMPSCI_760_Reinforcement/

